MEMORIA PRÁCTICA 3
CONVERSOR ANALÓGICO-DIGITAL

Patricia Cárdenas Fernández
3º Computadores A

ÍNDICE

1. MATERIALES Y HERRAMIENTAS ……. 2
2. FUNCIONAMIENTO Y MONTAJE ……. 2
3. ANOTACIONES DEL CÓDIGO ……. 3
3.1. NUESTROS PROTOTIPOS DE FUNCIONES Y VARIABLES PRIVADAS ……. 3
3.2. NUESTRO BUCLE WHILE ……. 4

1. MATERIALES Y HERRAMIENTAS

Para realizar esta práctica hemos utilizado un potenciómetro y cables. Estos se han utilizado para montar el circuito en una protoboard. Posteriormente, se ha utilizado un sensor de humo para ver que su comportamiento es exactamente igual.

Para tener mayor control sobre el motor necesitamos programar en C el comportamiento que queremos. Para ello hemos utilizado un microcontrolador **STM32 nucleo**, el cual lo hemos conectado al circuito.

Además, necesitamos configurar el microcontrolador para que sea capaz de suministrar las señales a los componentes del circuito. El programa **STM32CubeMX** nos facilita la configuración e inicialización de los pines que queremos usar. Una vez configurados los pines del microcontrolador, pasamos a programar el funcionamiento en un entorno de programación, como es **eclipse**.

2. FUNCIONAMIENTO Y MONTAJE

Un potenciómetro consta de un material resistivo, un cursor variable, una palanca giratoria y tres terminales. De hecho funciona como una resistencia variable y se usa como divisor de tensión. Para que el potenciómetro varíe su valor necesitamos alimentar una patilla a la fuente y otra a tierra, entonces giramos la palanca y obtenemos los valores de la resistencia. Para poder ilustrar dichos valores hemos utilizado el puerto serie del propio **STM32 nucleo** para conectarnos a una consola de datos.

Un ejemplo general de su funcionamiento es el siguiente:

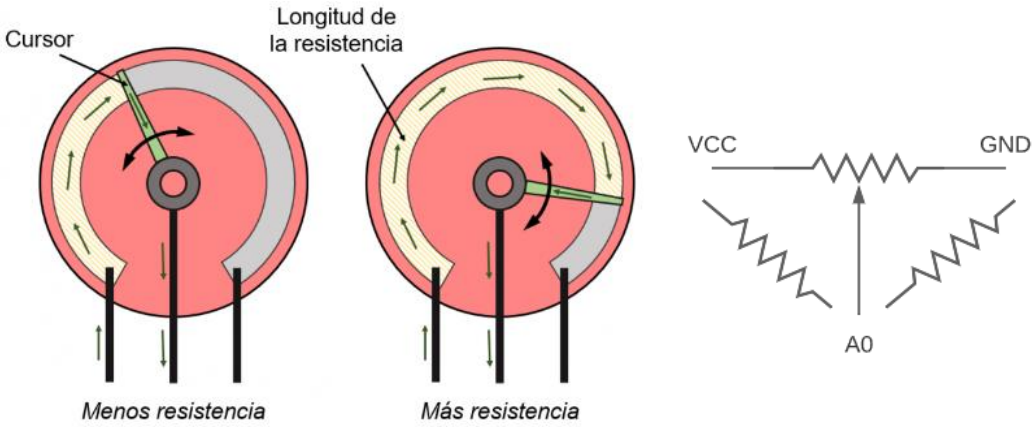


Ilustración 1. Funcionamiento del potenciómetro [1] y esquema

[1] https://www.yourtechnologyweb.com/3rd-eso-contents/electronics/3/

A continuación, se muestra un esquema del circuito. NOTA: Este esquema sólo sirve a modo de guía, ya que la placa que se usa es una **Arduino UNO** y en el laboratorio usamos la **STM32 nucleo**.

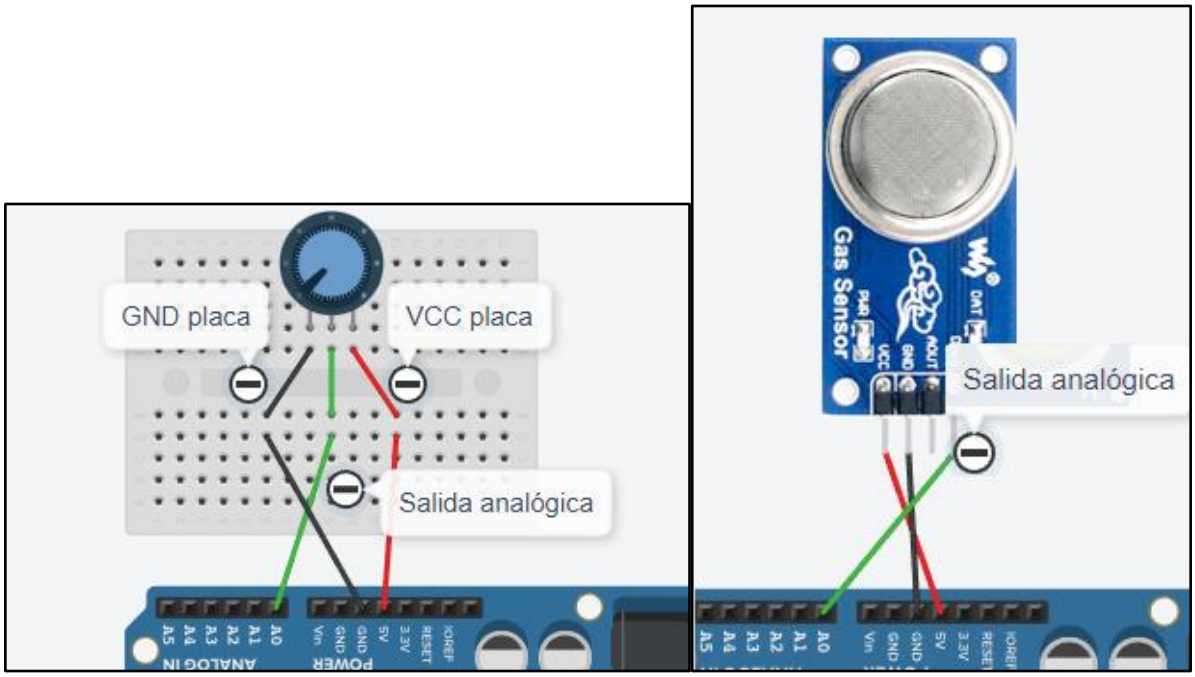


Ilustración 2. Montaje del potenciómetro (izquierda) y sensor (derecha) en placa protoboard

3. ANOTACIONES DEL CÓDIGO

En este apartado nos enfocamos en la programación, concretamente hablaremos un poco de cómo está estructurado el código. Es muy importante que escribamos nuestro código dentro de las secciones delimitadas por “USER CODE BEGIN” “USER CODE END” para no perder el código si volvemos a configurar el microcontrolador.

3.1. NUESTROS PROTOTIPOS DE FUNCIONES Y VARIABLES PRIVADAS

```
/* USER CODE BEGIN PFP */
size_t strlen(const char *str);
int sprintf (char *str, const char *format, ...);
/* USER CODE END PFP */
```

En este bloque del código definimos las funciones que posteriormente se implementarán. En este caso son funciones propias del lenguaje C, pero las estamos adecuando a nuestro uso.

Al igual que en el anterior, este bloque nos sirve para crear nuestras variables, por ejemplo, la cadena que contendrá el dato a mostrar por la consola y el valor analógico leído.

```
/* USER CODE BEGIN PV */
    uint8_t cadena[30];  //
    uint32_t valorADC;   //
/* USER CODE END PV */
```

3.2. NUESTRO BUCLE WHILE

```
/* USER CODE BEGIN WHILE */

// Este bucle while funciona como polling
while (1)
{

    // Comenzamos a convertir
    HAL_ADC_Start (&hadc1);

    // Obtenemos el valor del convertidor
    valorADC = HAL_ADC_GetValue (&hadc1);

    //Copiamos a la cadena
    sprintf ((char *)cadena, "valor = %lu \n\r", valorADC);

    // Necesitamos la longitud de los datos
    size_t tam = strlen ((char *)cadena);

    // Enviamos a UART
    HAL_UART_Transmit (&huart2, &cadena, tam, 1000);

    HAL_Delay (100);

  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Este será el bucle principal donde realizaremos toda la programación necesaria. Como se puede ver en los comentarios del código, este bucle nos sirve, a su vez, como polling.

En primer lugar se comienza la conversión y guardamos en **valorADC** el valor analógico leído. Posteriormente incluímos dicho valor en **cadena**, que será la variable que se imprima en la consola. Por último, usamos la **UART** para transferir esta cadena a través del puerto serie, y así mostrar los datos en la consola.